



Revolutionizing Application Modernization with AI

Krzysztof Kąkol
Lead Solutions Architect | Modernization & GenAI





Krzysztof Kąkol

Lead Solutions Architect in **Xebia**

AWS Community Builder & AWS Ambassador

<https://www.linkedin.com/in/krzysztofkakol/>

Other stuff:

Classical and jazz pianist
PhD in AI-driven sound processing

Presentation overview

The Challenge of Legacy Systems

How GenAI Transforms Code Modernization

GenAI-enhanced modernization on AWS

Real-life scenario

Sample use cases



The Challenge of Legacy Systems

75% of enterprise code is still in legacy languages (COBOL, Fortran, VB6, etc.)

42% of time wasted managing technical debt

Avg. technical debt cost: \$361,000 per 100,000 lines of code

The Challenge of Legacy Systems

Business Drivers

- Lack of skilled developers
- Cost of maintenance
- Performance & scalability issues
- Security & compliance risks
- Technical debt accumulation



How GenAI Transforms Code Modernization

- Automates code analysis
- Extracts non-existing documentation
- Reduces manual effort and error rates
- Removes “curse of the blank page”
- Reduces team frustration – allows to focus on interesting things



Key Benefits & Challenges



Key Benefits

- Code comprehension at scale
- Semantic translation (not just syntax)
- Documentation recovery
- SME knowledge extraction
- Safer & faster modernization pipelines



Challenges

- Hallucinations & verification challenges
- Regulatory & safety concerns
- Human-in-the-loop still critical

GenAI-supported Modernization

The new modernization workflow

- Code inventory + analysis
- LLM-powered transformation
- Human review + validation
- Continuous test-driven verification
- Deploy incrementally





Code Modernization with GenerativeAI

Where GenAI helps



Initial Investigation

GenAI speeds up initial investigation – it may detect technology, frameworks used, technical dependencies, and many more.

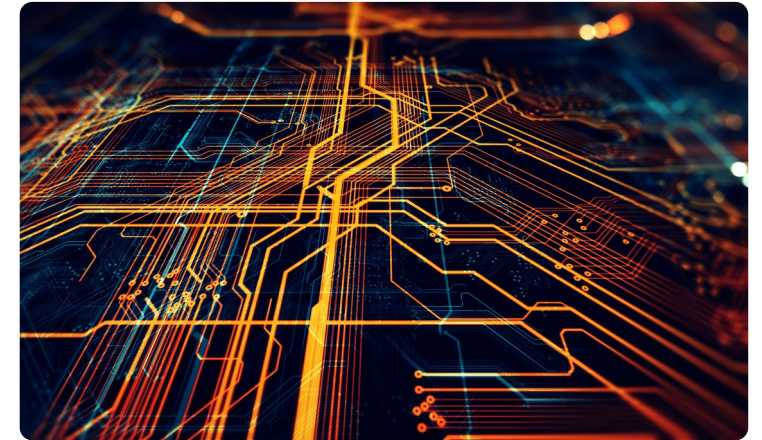
Therefore, the process of modernization can be started quickly without spending hours on initial code and tech stack analysis.



Preparing Documentation

Most of the legacy software has little to no documentation. It is crucial to understand the purpose of modernized modules or programs.

Such documentation can be automatically created, with a required level of detail – from an executive summary to a detailed description of certain classes or parts of code.



Detecting Technical Debt

Detecting technical debt is one of the most challenging parts in the modernization process. Many various factors must be taken into consideration (eg. security), and initial analysis might be difficult.

GenAI can be used to provide information about detected technical debt in minutes. This can be used to speed up further code and architecture analysis.

Where GenAI helps



Recommendations

Efficient modernization planning is a huge challenge. Multiple ideas must be evaluated against business requirements and expected outcomes.

GenAI may effectively help to take decisions by **accelerating the preparation of a set of recommendations** which can be discussed further.



Building Foundations

In complex projects, a lot of code is doing relatively repetitive tasks to keep the whole codebase maintainable and extendable. Writing such code when modernizing an application takes a lot of time.

GenAI may **write these parts (the code foundations)** for you. Therefore, teams may focus on business logics instead of doing boring, repetitive tasks.



Modernization Execution

There are various types of modernization journeys: from one database to another; migration to cloud; modernizing frameworks; changing technology etc.

GenAI can significantly **speed up modernization effort** by automatically converting parts of the legacy code to the new, selected technology.

GenAI-supported Modernization



Off-the-shelf tools

- Multiple migration tools like RhinoAI, IBM WatsonX
- Supporting tools – Copilot, RooCode, Cursor, Claude, Amazon Q
- Open-source LLM models fine-tuned for COBOL, ABAP, Fortran



Custom approach

- Utilizing foundation LLMs and prompt engineering
- Pattern matching
- Custom orchestration
- Full control and automation
- Using cloud-based models, with services like Amazon Bedrock

GenAI-supported Modernization

- Copilot
- Amazon Q
- Perplexity
- Cursor
- ...



Advantages:

- Solve a lot of day-to-day issues
- Enable IDE plugins

Disadvantages:

- Limited flexibility
- Little control on guardrails and system prompts
- Little automation capabilities
- Require a lot of skills to expand the possibilities of automation tools

Why custom tools might be better



Full control

We can decide how the system works under the hood. We can control:

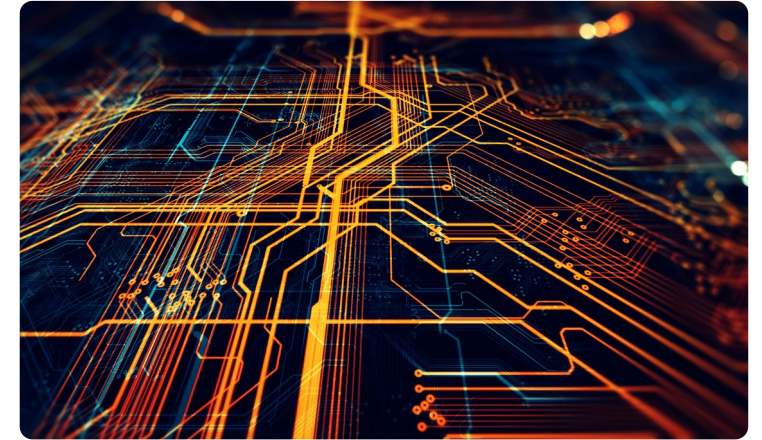
- Guardrails
- System prompts
- Internal system knowledge
- Pattern usage



Massive transformations

Custom tools might work on large codebases in parallel. We can for instance:

- Process all files at once
- Gather knowledge from many folders
- Work on the big code structure



Audit capabilities

We can audit the whole codebases and other sources. We can automate auditing processes and generate documents or reports on-demand.

Audtis can be technology-oriented, client-oriented, business-oriented etc. We can adapt the audit process to any external conditions.

Why custom tools might be better



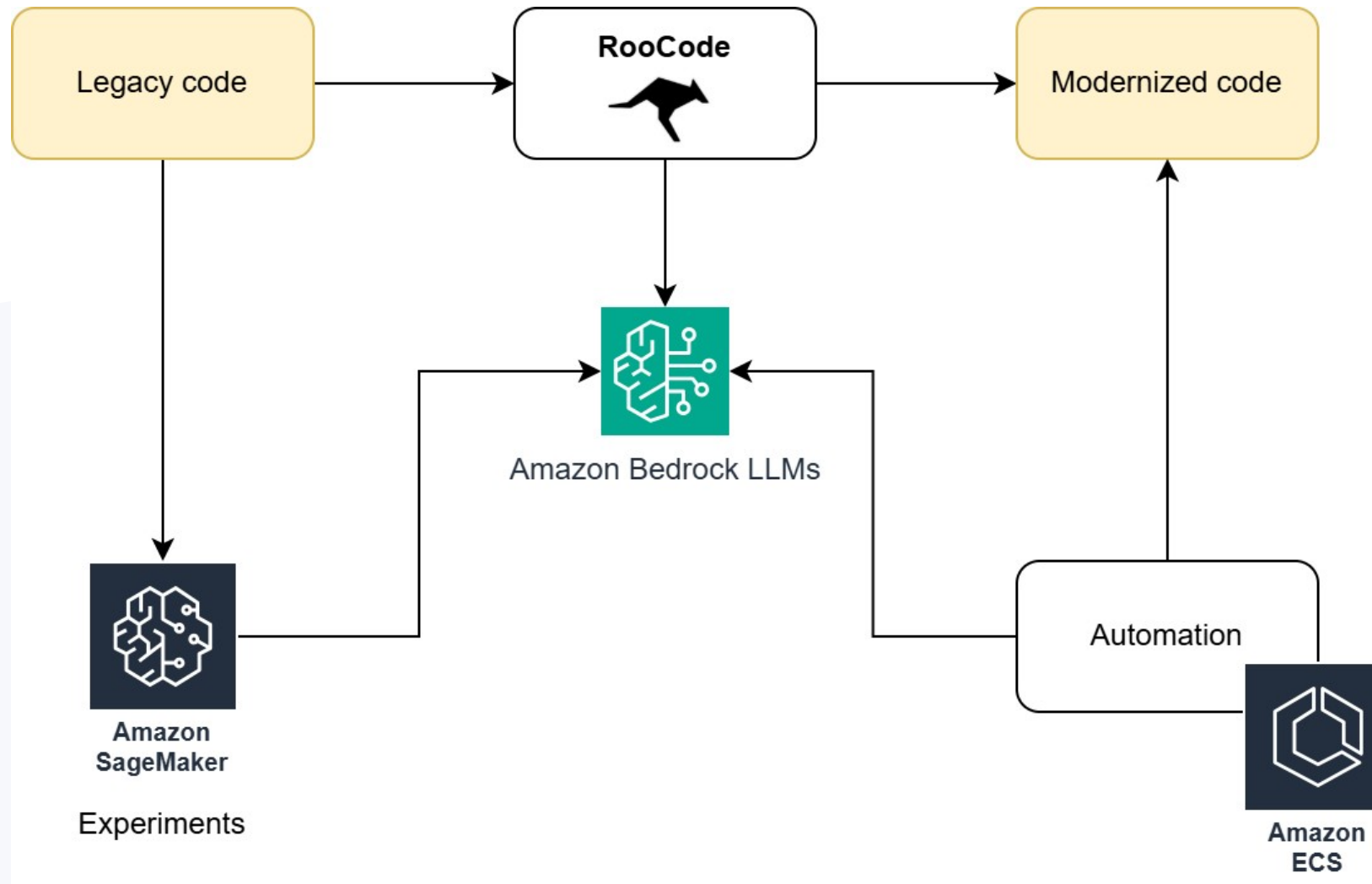
Process automation

We can use our tools to automate processes on development side. For instance, it's possible to create a customized hook in the CI/CD process or automate modernization process.

Amazon Services for GenAI Modernization

- **Amazon Bedrock** (Foundation Models - access to leading LLMs: Claude, Amazon, Cohere, Mistral, Llama etc.)
- **Amazon Q** (Code Transformation)
- **AWS Lambda, EKS, ECS** (Cloud Infrastructure – running transformation at scale)
- **Amazon SageMaker AI** (experimenting, running transformation endpoints)

Modernization architecture





Sample scenario

Modernizing a Legacy .NET Monolith with GenAI

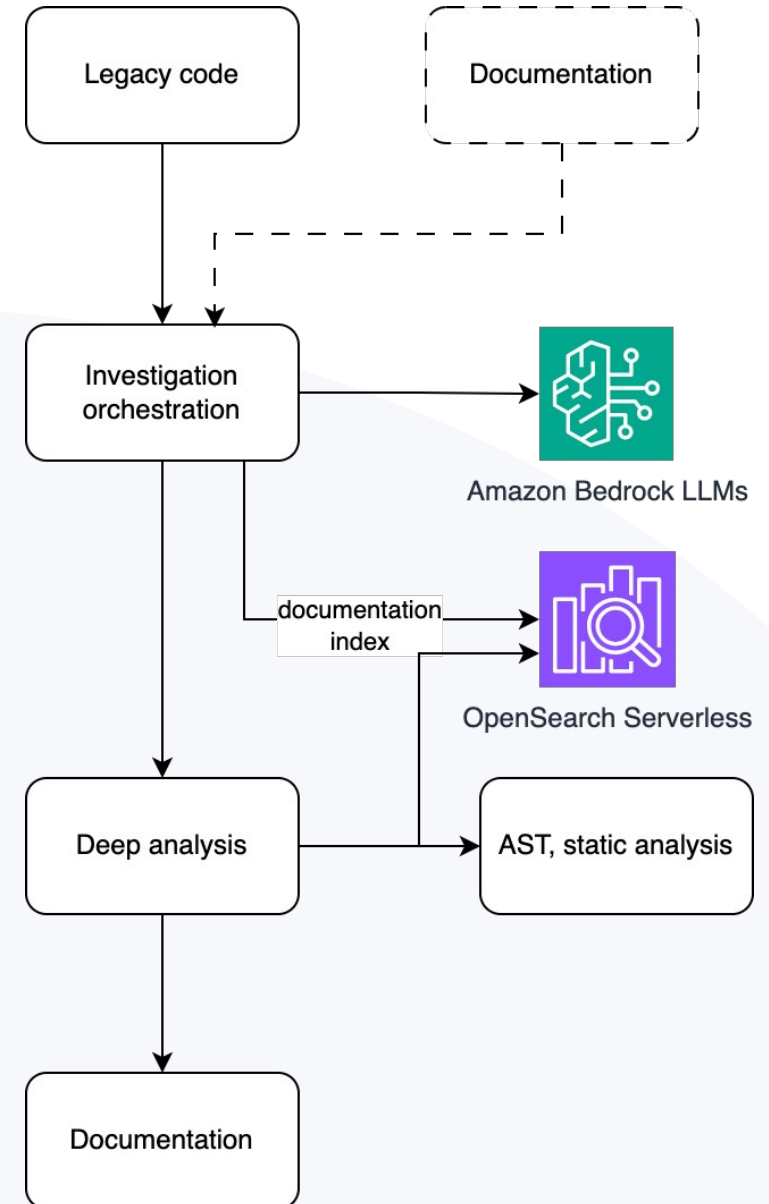
Input: Legacy .NET Monolith

"We have a 15-year-old ASP.NET Web Forms application, 600,000 lines of C# code. It manages customer orders, invoicing, and warehouse inventory."

- Tight coupling between UI, business logic, and data access
- Outdated architecture: no dependency injection, no microservices, no API layer
- Difficult to scale and maintain
- No documentation and no business requirements
- No tests at all

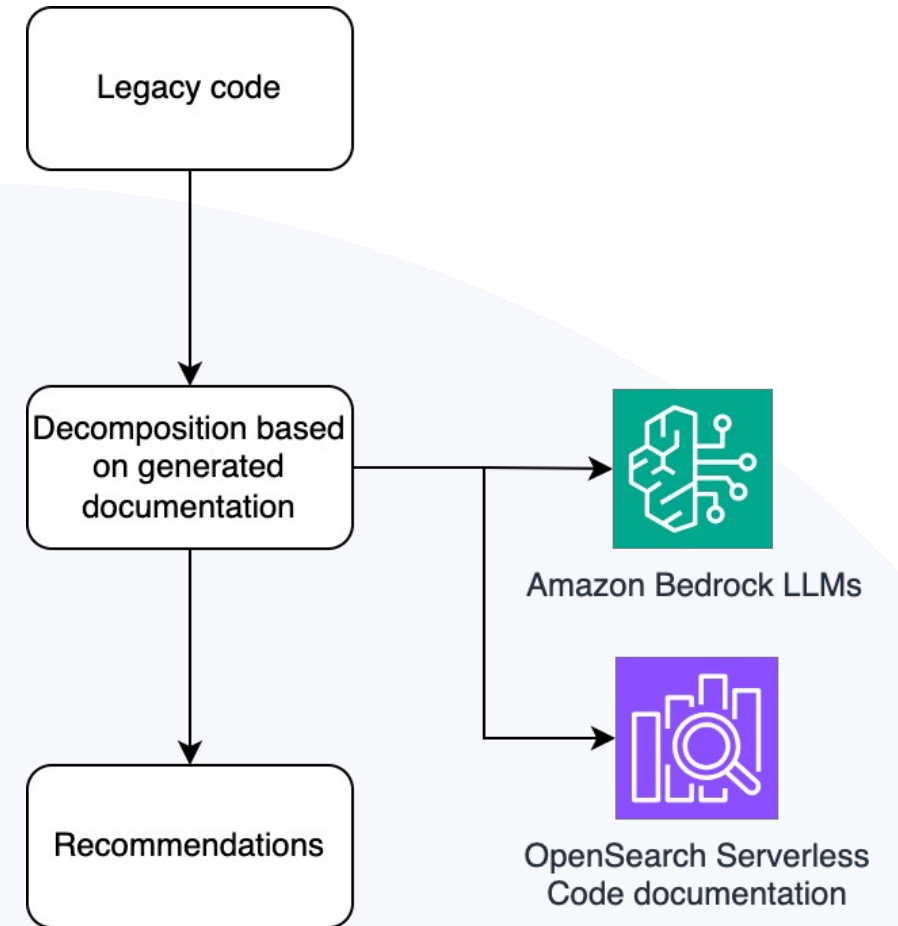
Initial Analysis

- **Code ingestion** - LLM reads the full C# codebase including:
 - Web Forms pages
 - Business logic classes
 - Stored procedures
 - Entity Framework or ADO.NET code
- **Dependency mapping** - GenAI generates:
 - Service boundaries
 - Data access patterns
 - Cross-cutting concerns (logging, security, etc.)
- **Hotspots identification** - Highlights tightly coupled modules, circular dependencies, and high-risk areas.



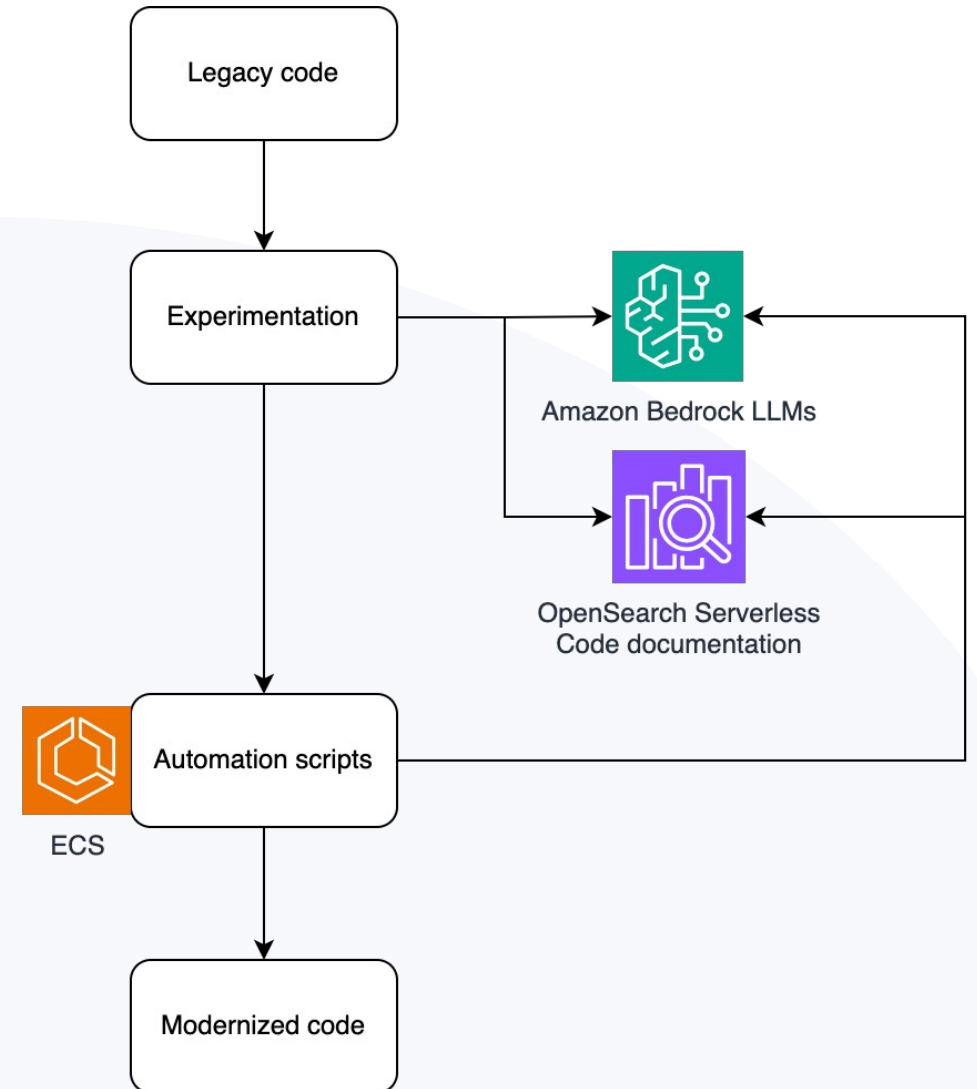
Knowledge Extraction

- **Business logic mapping** - Extracts core business rules into independent documentation
- **UI analysis** - Maps page flows, user interactions, form validations.
- **API candidates** - Identifies chunks of logic that can be refactored into REST endpoints.



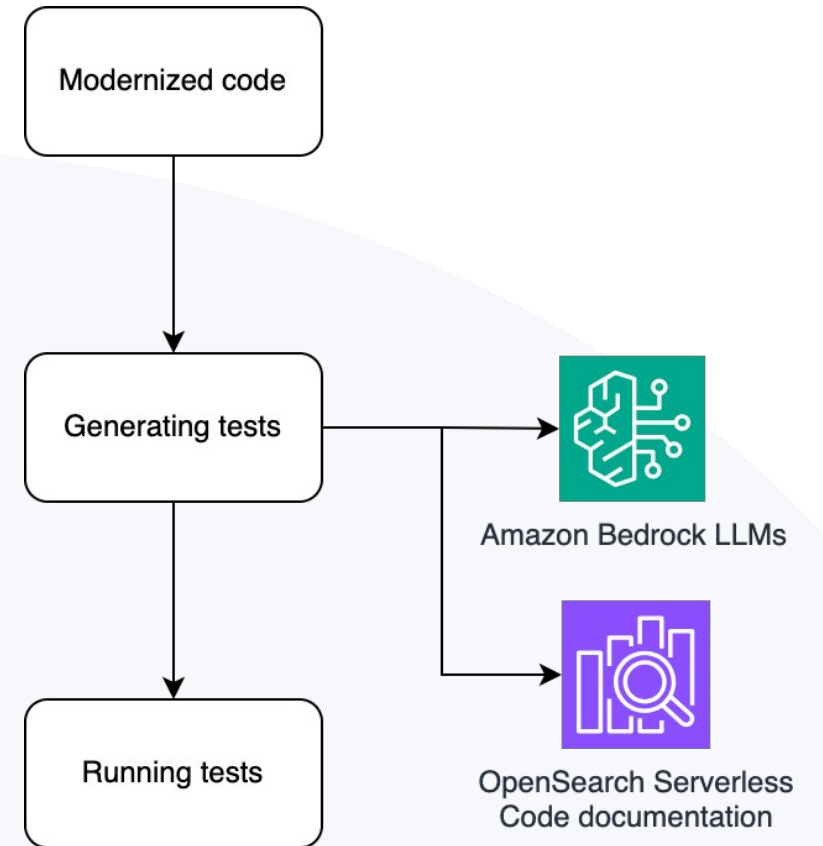
GenAI Transformation & Refactoring

- **Propose microservice boundaries** - Inventory, Orders, Billing, Customer Management.
- **Rewrite suggestions:**
 - ASP.NET Web Forms → ASP.NET Core MVC or Blazor
 - ADO.NET → EF Core or Dapper
 - Inline SQL → Repository pattern
- **Boilerplate code generation:**
 - Controllers, services, DTOs, DI containers
 - Swagger/OpenAPI specs for microservices
 - Auto-generated integration tests
- **Data layer modernization:**
 - Suggests breaking stored procedures into service methods.
 - Generates EF Core migrations where possible.



Human-in-the-Loop Review

- **Developers review** proposed boundaries.
- Code samples reviewed for **performance, security, and compliance**.
- Automated tests validate **functional equivalence**.





Done?

No! This process is iterative.



Use cases



Features migration in HRM application

- Features migration from old version of the HRM application written using custom PHP framework to the newest version of Symfony (PHP).
- Staged approach results in code boilerplates generation:
 - Entities
 - Repositories
 - API specification
 - ... and many more
- As a result, manual code migration efficiency was improved by around **20%**.

```

provided is a WP class named 'Wk_GIPS_Hours_Adapter_AdapterDatabase' is
for interacting with the database to fetch and save various data related
to GIPS.

Wk_GIPS_Hours_Adapter_AdapterDatabase.php:
1 Wk_GIPS_Hours_Adapter_AdapterDatabase
2 namespace Wk_GIPS_Hours_Adapter
3 {
4     class Wk_GIPS_Hours_Adapter_AdapterDatabase
5     {
6         /**
7          * Class Wk_GIPS_Hours_Adapter_AdapterDatabase
8          */
9         class Wk_GIPS_Hours_Adapter_AdapterDatabase extends Wk_GIPS_Hours_Adapter_Adapter
10         {
11             /**
12              * Keeps employee org unit IDs
13              *
14              * @var int[] = [EmployeeID => HomeOrgUnit]
15              */
16             protected static $HomeOrgUnit = [];
17
18             /**
19              * Retrieves GIPS_Calendar data
20              *
21              * @param int $EmployeeID
22              * @param PK_DateTime $From
23              * @param PK_DateTime $To
24              * @return stdClass[] = db.GIPS_Calendar[]
25              * @throws WPException
26              */
27             protected function fetchCalendar($EmployeeID, PK_DateTime $From, PK_DateTime $To)
28             {
29                 $sql = "SELECT *
30                     FROM GIPS_Calendar
31                     WHERE EmployeeID = :employeeid
32                     AND Date BETWEEN :from AND :to
33                     ORDER BY EmployeeID, Date, ";
34                 $params = [':employeeid' => (int)$EmployeeID, ':from' => $From->date, ':to' => $To->date];
35                 $res = PK_WkUtils::fetchAll($sql, false, $params);
36                 return $res;
37             }
38         }
39     }
40 }

```

Unit tests generation

- Modules of HR application contained a lot of non-tested code
- GenAI framework helped to generate unit tests for non-covered parts of code
- As a result, we saved around 80-90% of time needed to write unit tests

```

provided is a WP class named 'Wk_GIPS_Hours_Adapter_AdapterDatabase' which
interacting with the database to fetch and save various data related
to GIPS hours.

Wk_GIPS_Hours_Adapter_AdapterDatabase.php:
1 Wk_GIPS_Hours_Adapter_AdapterDatabase
2 namespace Wk\Adapter
3 {
4     <?php
5
6 /**
7  * Class Wk_GIPS_Hours_Adapter_AdapterDatabase
8  */
9 class Wk_GIPS_Hours_Adapter_AdapterDatabase extends Wk_GIPS_Hours_Adapter
10 {
11
12     /**
13      * Keeps employee org unit IDs
14      *
15      * @var int[] = [EmployeeID => HomeOrgUnit]
16      */
17     protected static $HomeOrgUnit = [];
18
19 /**
20  * Retrieves GIPS_Calendar data
21  *
22  * @param int $employeeID
23  * @param PK_DateTime $from
24  * @param PK_DateTime $to
25  * @return stdClass[] = db.GIPS_Calendar[]
26  * @throws \PDOException
27  */
28     protected function fetchCalendar($employeeID, PK_DateTime $from, PK_DateTime $to)
29     {
30         $sql = 'SELECT *
31             FROM GIPS_Calendar
32             WHERE EmployeeID = :employeeID
33             AND Date BETWEEN :from AND :to
34             ORDER BY EmployeeID, Date, '
35         $params = [':employeeID' => (int)$employeeID, ':from' => $from->toDateTime(), ':to' => $to->toDateTime()];
36         $res = PK_MysqlUtil::fetchAll($sql, false, $params);
37         return $res;
38     }
39 }
40 }

```

Stored procedures migration

- Accounting application has most of its business logics written in database in the form of stored procedures
- Stored procedures are hard to maintain and test
- Migration to .NET 8 is planned and we've made a POC which aim was to verify the feasibility of such migration
- Around 30-40% of the time might be saved in the modernization process

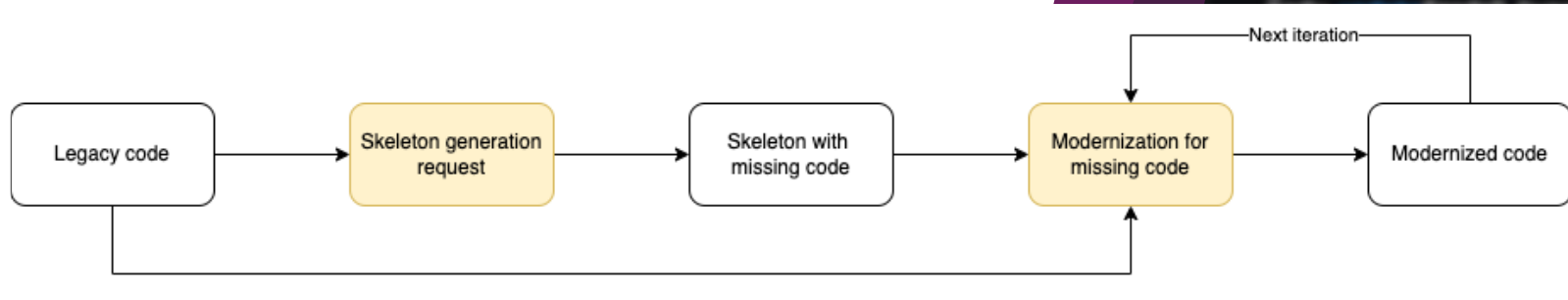
```
provided is a PHP class named 'WM_GDPH_Hours_Adapter_AdapterDatabase' which is used for interacting with the database to fetch and save various data related to the accounting application.
```

```
WM_GDPH_Hours_Adapter_AdapterDatabase.php
WM_GDPH_Hours_Adapter_AdapterDatabase
class WM_GDPH_Hours_Adapter_AdapterDatabase
{
    <?php
    //
    // Class WM_GDPH_Hours_Adapter_AdapterDatabase
    //
    class WM_GDPH_Hours_Adapter_AdapterDatabase extends WM_GDPH_Hours_Adapter_AdapterDatabase
    {
        //
        // keeps employee org unit ID
        //
        // @var int[] - [EmployeeID => HierarchyID]
        //
        protected static $HierarchyID = [];

        //
        // Retrieves GDPH_Calendar data
        //
        // @param int EmployeeID
        // @param PK_DateTime $from
        // @param PK_DateTime $to
        // @return stdClass[] - db.GDPH_Calendar[]
        // @throws \PDOException
        //
        protected function fetchCalendar(EmployeeID, PK_DateTime $from, PK_DateTime $to)
        {
            $sql = 'SELECT *
            FROM GDPH_Calendar
            WHERE EmployeeID = :employeeid
            AND Date_ BETWEEN :from AND :to
            ORDER BY EmployeeID, Date_';
            $params = ['employeeid' => $EmployeeID, ':from' => $from->date, ':to' => $to->date];
            $res = $this->getDBConnection()->query($sql, $params);
            return $res;
        }
    }
}
```

Legacy code modernization for accounting application

- Legacy code modernization from ASP.NET code to .NET 6.
- Staged approach results in full migration to the new platform. All necessary components were auto-generated and the modernization process requires some refactoring, code validation and testing.
- Overall, the modernization process efficiency may be improved by around **20%**.



```
...List("bedrock-runtime")
...content(path)
...
...in path
...files = [path for f in os.listdir(path) if os.path.isfile(path)]
...listfiles = listfiles + files
...
...filepath in listfiles
...with open(filepath) as f: s = f.read()
...
... = ["../M code/salarytype/entity", "../M code/salarytype/repo", "../M code/MODEL"]
get_code_content(path)
get_model_result(prompt_data, model_id="anthropic.claude-3-haiku-202403-07-v1:0")
model_usage = {
    "api_tokens": 1000, "temperature": 0.1,
    "top_p": 0.5, "top_k": 1, "stop_sequences": ["\n\n"],
}
# Input configuration
prompt = prompt_data
body = {
    "anthropic_version": "bedrock-2023-05-01",
    "system": "You are a honest and helpful bot.",
    "messages": [
        {"role": "user", "content": [{"type": "text", "text": prompt}],
    ],
    1,
}
```

Windows-based WPF application modernization

- Old WPF application (Windows app) should be migrated to web-based Blazor framework
- Migrating legacy WPF forms to Blazor-based forms is done with the help of our GenAI framework
- Initial work is fully done by our framework, which helps to save between 20-30% of the time needed to migrate

```
def client_invoke_model(client, model_id, prompt_data, body):
    response = client.invoke_model(
        model_id=model_id,
        body=json.dumps(body),
    )
    # Process and print the response
    result = json.loads(response.get("body").read()).get("content", None).get("text", "")
    return result

def get_model_content(paths):
    # Get the content of the files
    files = []
    for path in paths:
        if os.path.isfile(path):
            files.append(path)
    # Read the content of the files
    with open(files[0]) as f:
        s = f.read()
    return s

def get_model_result(prompt_data, model_id="anthropic.claude-3-haiku-20240307-v1:0"):
    model_params = {
        "max_tokens": 1000, "temperature": 0.1,
        "top_p": 0.9, "top_k": 1, "stop_sequences": ["\n\nHuman:"],
    }
    # Input configuration
    prompt = prompt_data
    body = {
        "anthropic_version": "bedrock-2023-05-01",
        "system": "You are a honest and helpful bot.",
        "messages": [
            {
                "role": "user", "content": [{"type": "text", "text": prompt}],
            }
        ],
    }
    body.update(model_params)

    response = client_invoke_model(
        model_id=model_id,
        body=json.dumps(body),
    )
    # Process and print the response
    result = json.loads(response.get("body").read()).get("content", None).get("text", "")
    return result
```

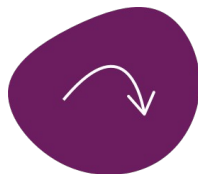
The Challenge of Legacy Systems

Traditional modernization

- Manual rewrites
- Re-architecture
- High risk, long timelines

GenAI supported modernization

- Code understanding at scale
- Automated refactoring
- Lower risk, less frustration, shorter timelines



Questions?



<https://www.linkedin.com/in/krzysztofkaKol/>

